内 容 安 排

章	内 容	学时	章	内 容	学时
1	绪 论	2	7	图	5
2	线性表	5	8	动态存储管理	略
3	栈和队列	3	9	查找	3
4	串	3	10	内部排序	5
5	数组和广义表	3	11	外部排序	略
6	树和二叉树	7	12	文件	略

第5章　数组和广义表
（Arrays & Lists）

5.1 数组的定义

5.2 数组的顺序存储表示和实现

5.3 矩阵的压缩存储

5.4 广义表的定义

5.5 广义表的存储结构

5.1 数组的定义

数组： 由一组名字相同、下标不同的变量构成

注意： 这里讨论的数组与高级语言中的数组有所区别：高级语言中的数组是顺序结构；而这里的数组既可以是顺序的，也可以是链式结构，用户可根据需要选择。

判断：“数组的处理比其它复杂的结构要简单”，对吗?

答：对的。因为——

① 数组中各元素具有统一的类型；

② 数组元素的下标一般具有固定的上界和下界，即数组一旦被定义，它的维数和维界就不再改变。

③数组的基本操作比较简单，除了结构的初始化和销毁之外，只有存取元素和修改元素值的操作。

一维数组的特点： 1个下标，a_i 是a_{i+1}的直接前驱

二维数组的特点： 2个下标，每个元素$a_{i,j}$受到两个关系（行关系和列关系）的约束：

$$A_{mn}=\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

一个m×n的二维数组可以看成是m行的一维数组，或者n列的一维数组。

N维数组的特点： n个下标，每个元素受到n个关系约束

一个n维数组可以看成是由若干个n－1维数组组成的线性表。

N维数组的数据类型定义

$n_ARRAY = (D, R)$ 其中：

数据对象：$D = \{a_{j1,j2\ldots jn} \mid j_i$为数组元素的第i 维下标， $a_{j1,j2\ldots jn} \in Elemset\}$

数据关系：$R = \{ R1, R2, \ldots. Rn \}$

$$Ri = \{<a_{j1,j2,\ldots ji\ldots jn}, a_{j1,j2,\ldots ji+1\ldots jn}> \mid a_{j1,j2,\ldots ji\ldots jn}, a_{j1,j2,\ldots ji+1\ldots jn} \in D \}$$

基本操作：**构造数组、销毁数组、读数组元素、写数组元素**

数组的抽象数据类型定义略，参见教材P90

5.2 数组的顺序存储表示和实现

问题： 计算机的存储结构是一维的，而数组一般是多维的，怎样存放?

解决办法： 事先约定按某种次序将数组元素排成一列序列，然后将这个线性序列存入存储器中。

例如： 在二维数组中，我们既可以规定按行存储，也可以规定按列存储。

注意：

- 若规定好了次序，则数组中任意一个元素的存放地址便有规律可寻，可形成地址计算公式；
- 约定的次序不同，则计算元素地址的公式也有所不同；
- C和PASCAL中一般采用**行优先**顺序；FORTRAN采用**列优先**。

无论规定行优先或列优先，只要知道以下三要素便可随时求出任一元素的地址（意义：数组中的任一元素可随机存取）：

①开始结点的存放地址（即基地址）
②维数和每维的上、下界；
③每个数组元素所占用的单元数

$$A_{mn}=\begin{bmatrix} a_{c1,c2} & \cdots & a_{c1,d2} \\ \cdots & a_{ij} & \cdots \\ a_{d1,c2} & \cdots & a_{d1,d2} \end{bmatrix}$$

补充：计算二维数组元素地址的通式

设一般的二维数组是$A[c_1..d_1, c_2..d_2]$，这里c_1,c_2不一定是0或1

则行优先存储时的地址公式为：

$$LOC(a_{ij})=LOC(a_{c_1,\ c_2})+[(i-c_1)*(d_2-c_2+1)+j-c_2)]*L$$

数组基址

a_{ij}之前的行数

总列数，即第2维长度

a_{ij}本行前面的元素个数

单个元素长度

二维数组列优先存储的通式为：

$$LOC(a_{ij})=LOC(a_{c_1,\ c_2})+[(j-c_2)*(d_1-c_1+1)+i-c_1)]*L$$

例1：如何求出a(3, 2)的存储地址？

	0	1	2	3
0	a(0,0)	a(0,1)	……	a(0,3)
1	a(1,0)	a(1,1)	……	a(1,3)
2	……	……	……	……
3	……	……	**a(3,2)**	……
4	……	……	……	……
5	……	……	……	……
6	a(6,0)	……	……	a(6,3)

要事先确定：

①是行优先方式还是列优先方式？

②数组的首地址是多少？

③每个元素的长度？

否则无法求出结果

例2〖软考题〗：一个二维数组A，行下标的范围是1到6，列下标的范围是0到7，每个数组元素用相邻的6个字节存储，存储器按字节编址。那么，这个数组的体积是_288_个字节。

答：Volume=m*n*L=(6-1+1)*(7- 0 +1)*6=48*6=288

例3：已知二维数组$A_{m,m}$按行存储的元素地址公式是：$Loc(a_{ij})= Loc(a_{11})+[(i-1)*m+(j-1)]*K$，请问按列存储的公式相同吗？

答：尽管是方阵，但公式仍不同。应为：

$$Loc(a_{ij})=Loc(a_{11})+[(j-1)*m+(i-1)]*K$$

例4：〖00年计算机系考研题〗：设数组a[1…60, 1…70]的基地址为2048，每个元素占2个存储单元，若以列序为主序顺序存储，则元素a[32,58]的存储地址为__8950_____。

答：请注意审题！

根据列优先公式 $Loc(a_{ij})=Loc(a_{11})+[(j-1)*m+(i-1)]*K$

得：$LOC(a_{32,58})=2048+[(58-1)*60+(32-1)]*2 = 8950$

想一想：若数组是a[0…59, 0…69]，结果是否仍为8950?

维界虽未变，但此时的a[32, 58]不再是原来的a[32, 58]

若是N维数组，其中任一元素的地址该如何计算?

教材已给出低维优先的地址计算公式（见P93（5-2）式）

该式称为n维数组的映像函数：

$$Loc(j_1,j_2,\ldots j_n)=LOC(0,0,\ldots 0)+\sum_{i=1}^{n} C_i j_i$$

数组基址

前面若干元素占用的地址字节总数

其中$C_n=L$, $C_{i-1}=b_i\times C_i$, $1<i\leq n$

每个元素长度

第i维长度

与所存元素个数有关的系数，可用递推法求出

三维数组且列优先时的元素地址要会计算！

例5：【专科考研资格考试】假设有三维数组$A_{7\times9\times8}$，每个元素用相邻的6个字节存储，存储器按字节编址。已知A的起始存储位置（基地址）为1000，末尾元素A[6][8][7]的第一个字节地址为多少？若按高地址优先存储时，元素A[4][7][6]的第一个字节地址为多少？

答：

① 末尾元素A[6][8][7]的第1个字节地址＝

1000 +(7×9×8)×6－6=4018

② 按高地址优先存储时，元素A[4][7][6]的第1个字节地址＝?

提示：将第3维看作“页码”，前面两维就是每页上的二维数组。

A[4][7][6]=1000+(7 ×9 ×6 +7 ×7+4) ×6= 3586

(高维地址计算通式参见清华殷人昆教材的解释)

计算地址的意义：

只要计算出任一数组元素的地址，就能对其轻松地进行读写操作！

N维数组的顺序存储表示（见教材P93)

```
#define MAX_ARRAY_DIM   8   //假设最大维数为8
typedef struct{
    ELemType *base;       //数组元素基址
    int        dim;       //数组维数
    int       *bound;     //数组各维长度信息保存区基址
    int       *constants; //数组映像函数常量的基址
}Array;
```

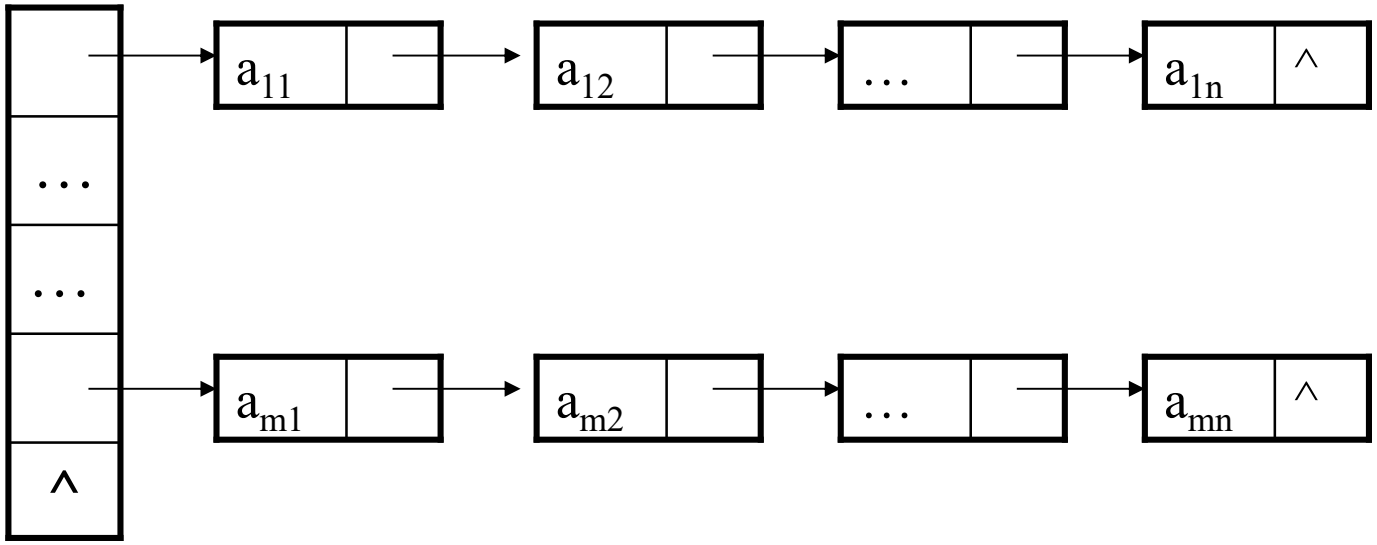
即C_i信息保存区

数组的基本操作函数说明（有5个）
(请阅读教材P93-95)

注意： 本章所讨论的数组与高级语言中的数组有所区别：高级语言中的数组只是顺序结构；而本章的数组既可以是顺序的，也可以是**链式结构**，用户可根据需要选择。

补充：数组的链式存储方式—用**带行指针向量的单链表**来表示。

行指针向量



注：**链式数组的运算请参见“稀疏矩阵的转置”**

5.3 矩阵的压缩存储

讨论：

1. 什么是压缩存储?

若多个数据元素的值都相同，则只分配一个元素值的存储空间，且零元素不占存储空间。

2. 所有二维数组（矩阵）都能压缩吗?

未必，要看矩阵是否具备以上压缩条件。

3. 什么样的矩阵具备以上压缩条件?

一些特殊矩阵，如：对称矩阵，对角矩阵，三角矩阵，稀疏矩阵等。

4. 什么叫稀疏矩阵?

矩阵中非零元素的个数较少（一般小于5%）

重点介绍稀疏矩阵的压缩和相应的操作。

一、稀疏矩阵的压缩存储　　二、稀疏矩阵的操作

问题：

如果只存储稀疏矩阵中的非零元素，那这些元素的位置信息该如何表示?

解决思路：

对每个非零元素增开若干存储单元，用来存放其所在的行号和列号，便可准确反映该元素所在位置。

实现方法：

将每个非零元素用一个三元组（i，j，a_{ij}）来表示，则每个稀疏矩阵可用一个三元组表来表示。

例1： 三元素组表中的每个结点对应于稀疏矩阵的一个非零元素，它包含有三个数据项，分别表示该元素的__行下标__、__列下标__和__元素值__。

例2： 写出右图所示稀疏矩阵的压缩存储形式。

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

解： **至少有4种存储形式。**

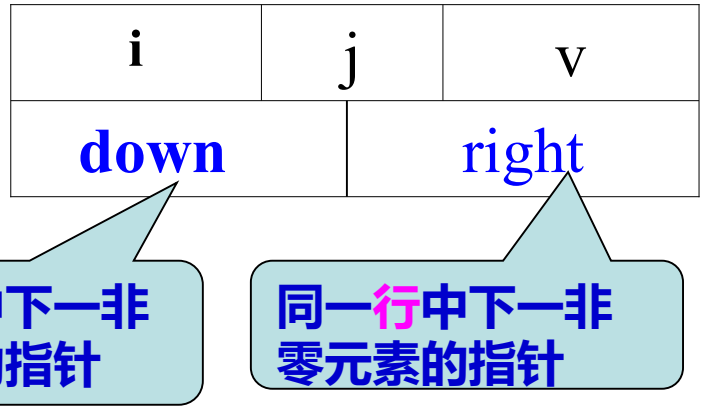
法1： **用线性表表示：**

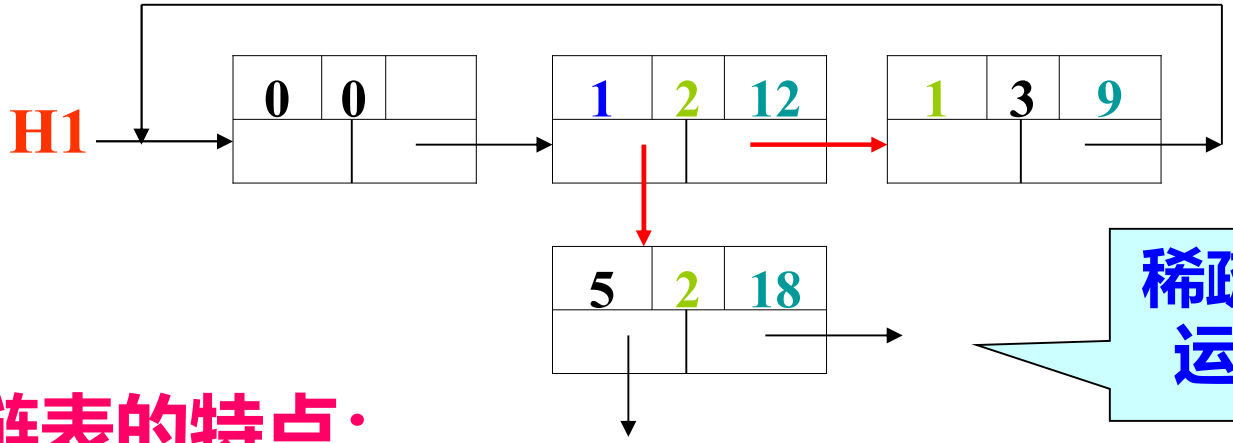
((1,2,12) ，(1,3,9)， (3,1,-3)， (3,5,14)，
(4,3,24)， (5,2,18) ，(6,1,15)， (6,4,-7))

法2：用十字链表表示

用途： 方便稀疏矩阵的加减运算

方法： 每个非0元素占用5个域





稀疏矩阵的加减运算容易实现

十字链表的特点：

①每行非零元素链接成带表头结点的循环链表；

②每列非零元素也链接成带表头结点的循环链表。

则每个非零元素既是行循环链表中的一个结点；又是列循环链表中的一个结点，即呈十字链状。

法3：用三元组矩阵表示：

	i	j	value
0	6	6	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	5	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

注意：为更可靠描述，通常再加一行“总体”信息：即总行数、总列数、非零元素总个数

稀疏矩阵压缩存储的缺点：

将失去随机存取功能 ！

例3：下面的三元组表表示一个稀疏矩阵，试还原出它的稀疏矩阵。

	i	j	value
0	6	4	6
1	1	2	2
2	2	1	12
3	3	1	3
4	4	4	4
5	5	3	6
6	6	1	16

$$\begin{pmatrix} 0 & 2 & 0 & 0 \\ 12 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 \\ 0 & 0 & 6 & 0 \\ 16 & 0 & 0 & 0 \end{pmatrix}$$

三元组表的顺序存储表示（见教材P98）

```
typedef struct{
 Triple data[MAXSIZE+1]; //三元组表，以行为主序存入一
                         维向量 data[ ]中
 int mu;                 //矩阵总行数
 int nu;                 //矩阵总列数
 int tu;                 //矩阵中非零元素总个数
}TsMatrix;
```

对三元组表data[ ]的整体定义

```
#define MAXSIZE 125000  //设非零元素最大个数
                         125000

 typedef struct{
 int i;                  //元素行号
 int j;                  //元素列号
 ElemType e;             //元素值
}Triple;
```

对表中每个结点的结构定义

法4：用带辅助向量的三元组表示。

用途：便于高效访问稀疏矩阵中任一非零元素。

方法：增加2个辅助向量：

① 记录每行非0元素个数，用NUM（i）表示；

② 记录稀疏矩阵中每行第一个非0元素在三元组中的行号，用POS（i）表示。

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

i	1	2	3	4	5	6
NUM(i)	2	0	2	1	1	2
POS(i)	1	3	3	5	6	7

	i	j	v
0	6	6	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	5	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

POS（i）如何计算？

用途后述

POS(1)＝1

POS(i)＝POS(i-1)+NUM(i-1)